

Claude Code: agentic coding versus Chat and Cowork

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 3 of 12 · English notes

After the cert map, Vijay switched to the product this course exists for: **Claude Code**, Anthropic’s AI-powered coding tool. It started as an assistant and is now an **agentic** client: it reads a whole project (RAG over the workspace), edits many files, runs terminal commands, tests, git/PRs, and talks to Jira, Figma, and other tools. It is not “Chat with a folder.” It is not Cowork’s invoice sorter. It is the SDLC agent.

Assistant versus agent (the contrast he drew)

Old coding assistant	Claude Code as agent
Suggests in the current file; waits for the next keystroke	Understands the repo; reads/modifies multiple files
Limited context, easy to lose the plot	Retrieves relevant chunks; can plan then execute
Recommends; you always paste	Takes actions with your permission: shell, tests, PRs
GitHub Copilot’s early identity in his story	Copilot has also moved toward agents — still not this product

A single prompt can now aim at backend, frontend, database scripts, Docker, Terraform, Jenkins — a deployable workspace. That is the productivity claim: less repeated toil, faster sprints, cleaner formatting if you encode standards. Existing brownfield repos still need good prompts (Lalit’s question); greenfield boilerplate is the easy demo.

How the loop runs

1. User prompt — intent might be “plan only” or “build the project.”
2. Selected model (Sonnet / Opus / Haiku / others your org enables) interprets intent.
3. Explore the project for context.
4. Create an execution plan.
5. Execute with built-in tools: read/edit files, shell, third-party MCP/connectors, git.
6. Verify (e.g. read terminal logs as I/O tools) and either finish or ask you.

Terminal logs are a **tool**: the agent reads success/failure and continues. That is the same plan-versus-execute split as Cowork, with the filesystem and compiler as hands.

Install paths he demonstrated

Prerequisites: Anthropic console login on Pro/Max/Team/Enterprise; **Node.js 18+**; a terminal. Optional: VS Code (he preferred it over Claude Desktop’s editor because extensions make the environment real). Two upstream installs: vendor `curl` script or `npm` global. After install, the CLI command is `claude` in the folder

that should be the workspace. Login/authorize through the console. Security notes on first run: models make mistakes; prompt-injection risk — only use on code you trust. Beta bits are not in the default installer; UST will want portal/private-registry install, not random curl on a production laptop. Windows: people in chat had to put Claude on PATH / environment variables.

CLI then VS Code

First-run wizard: terminal theme, trust this git folder, show version / user / default model. Slash commands already appear: `/init` for CLAUDE.md, `/release-notes`, `/model`, `/exit`. Then he installed the **Anthropic** VS Code extension (not a clone publisher), authorised Claude.ai, and used the Code icon for sessions. Plus button = new session; attach open files, PDFs, web search; some features with a terminal glyph still drop you to the integrated terminal because the UI has not caught up.

UI VERSUS DOCS

Extension IDs, default models (he had Sonnet 5 selected in class), and which slash commands live only in the TTY will change. Prefer **Claude Code docs** the week you sit the exam.

When to pick Code instead of Chat or Cowork

- **Chat:** explain an algorithm, draft an email, no repo.
- **Cowork:** shuffle local office files; non-dev multi-step jobs.
- **Code:** the workspace is source control; you need tests, PRs, clean-code automation, or generation of the app itself.

Exam angle

If the stem is a developer who must implement, debug, or wire CI, the named product is Claude Code (or the API), not Cowork. If the stem is “organise PDFs on disk for a BA,” do not show off Code. Product fit is a Foundation skill.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Install, CLI, IDE — the source of truth.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

What `/model` can point at in your tenant.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Shell and file tools are this idea inside Code.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Jira, Figma, and custom servers from the agent.

- **API overview**

<https://docs.claude.com/en/api/overview>

When the answer is “don’t use the GUI agent; call Messages.”

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.